

Greedy Algorithms

Introduction to Greedy Algorithms

Algorithm for an optimization problem goes through a sequence of steps with a set of choices at each step. For many optimization problems, using dynamic programming to determine the best choice is overkill. A Greedy Algorithm always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.

Greedy algorithms do not always yield optimal solutions, but for many problems they do.

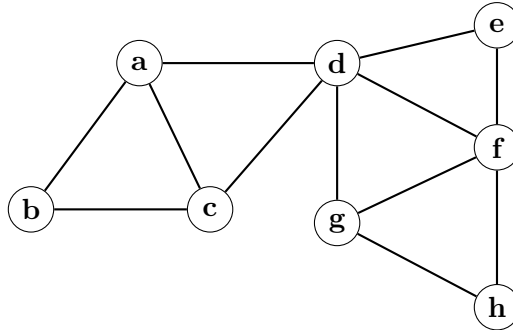
Example

Scheduling several competing activities that require exclusive use of a common resource.

- **Goal:** Selecting a maximum sized set of mutually compatible activities.
- **Suppose:** $S = \{a_1, a_2, \dots, a_n\}$ of n proposed activities wish to use a resource. Each activity a_i has a start time s_i and finish time f_i where $0 \leq s_i < f_i < \infty$.

Greedy Algorithm - 1st Example

Independent set: in a graph G is a subset U of the vertex set $V(G)$ such that for any $x, y \in U$, $(x, y) \notin E(G)$.



Example Independent sets:

- $\{a, e, f\}$
- $\{a, c\}$
- $\{e, h\}$

These are all independent sets. We have listed out a few such Independent sets but there are many others.

Each vertex by itself is an Independent set. So formally, an Independent set is a collection of vertices so that there are no edges between them.

Applications: Example of a Radio Station. There are other places where one can look for an Independent set. There are many other applications.

Maximum Independent Set on Trees

Well, the general problem is: given a graph, we want to find an Independent set of large size. This problem turns out to be hard and we will look at it later in the course.

For now, we restrict the graph to be trees. So the problem now becomes:

- **Input:** A tree T (A tree is a graph which is connected and acyclic).
- **Output:** Maximum sized Independent set in T . I want a subset of the vertex set in T of maximum size such that there is no edge between any pair of vertices.

So how do we find a maximum sized Independent set in T ?

Suppose T has n vertices. One way to do it is to sort of use a brute force approach, which looks for all possible subsets of the vertex set, looks at whether they are an Independent set and chooses the one which has maximum size.

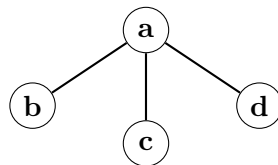
But this approach is too expensive; the time taken is at least the number of subsets of the vertices, that is 2^n . We want to do it much faster.

Well, what else one could do is pick a vertex, and once it is in the independent set then the neighbors of it will not be included in the Independent set, so that is by definition of the Independent Set.

So, pick a vertex. Throw out its neighbors. Look at the graph again, pick a vertex that remains, throw out its neighbors and so on.

This is fine, but how do you ensure that the independent set that is found has maximum size? If you pick a wrong vertex at any point you will be in trouble.

So here is a simple example:



Consider this graph. If we pick vertex a , then we get an independent set of size one, whereas the optimum is of size three (picking b, c, d).

The solution to the problem is a subset of the vertex set and you have to somehow pick the right subset. And the greedy way says pick one by one somehow.

Now which vertex would you like to put in the Independent set so that it becomes large? Intuition tells you that you should pick a vertex of smallest degree in this graph.

If you pick a vertex of small degree, then the number of vertices you throw out will be smaller. In particular, if we pick a leaf node in the independent set, then we throw out just only one vertex, and this actually works in the sense that it actually produces a maximum size Independent set.

Proving the Greedy Strategy

We need a proof that this actually produces a maximum size Independent set.

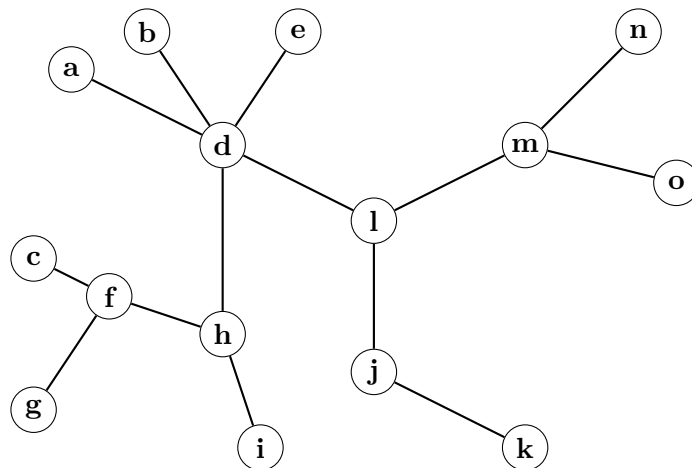
The algorithm is just this:

1. Pick a leaf.
2. Throw out its neighbors.
3. Continue.

At each stage we will have a forest. If we have a vertex of degree zero, it is not adjacent to any other vertex which we will come across as the algorithm proceeds, put this vertex in the Independent set blindly.

If no such vertex exists then we need to pick a leaf (many times as the algorithm proceeds) and put it in the Independent set and continue.

This algorithm works, however it needs a proof. The algorithm is greedy in the sense that we started with a piece and have a part of the solution and do not change it. We add new vertices to it and never leave.



Another way to look at the problem and the same algorithm. Find Max Independent set in this tree given.

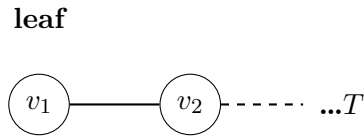
Suppose we have some subset which is an Independent set, $S = \{a, e, f, i, k, m\}$. The algorithm has not picked it. Suppose somebody has guessed this set. This is to get your thinking caps going.

Now the question, can I add and subtract to this solution to increase the size? If you look carefully, we can add n to S and remove m . From the example actually it looks like we can always do this with the leaf.

So this naturally raises the question that, do all leaves always fall in some optimum Independent set?

The answer is yes. Why?

Well supposing we have a tree and a leaf as follows:



One of these has to better be in the Independent set. Because if v_1 is not in the IS, then we can add v_1 .

So suppose v_2 is in the IS, then we can add v_1 to the IS and remove v_2 . The size still does not change. The set is an IS and the size does not change.

We picked one element out and put one element in and the size does not change. This is the exchange argument we needed for the IS.

This is a one line proof that all leaves should be in some optimum IS.

So we can sort of by putting all leaves in the IS as there is an optimum that contains these leaves. Remove the neighbors and recurse on the remaining.

This is the good thing about a greedy algorithm: if you pick a part of the output, then you have a smaller input to deal with and you can apply the same technique over and over again.

Algorithm Summary:

1. Put all the leaves in the IS.
2. Remove their neighbors.
3. The tree will now have many disconnected parts (forest). Recurse on each part.